intelligent enterprise
SMARTER, FASTER, MORE PROFITABLE



View Content | Search | Sort by Date Descending | GO | Advanced Options

Communities
Meet colleagues and get news, events information, and other resources

IntelligentCRM
Customer relationship management strategies for competitive advantage

IntelligentEAI
Technologies and business strategies for enterprise application integration

IntelligentERP
News, events, and discussion customized for managers of ERP systems

IntelligentKM
Knowledge management and its role in the extended, intelligent enterprise
Sponsored by iManage

Information Centers
Topic-oriented views of IE articles

Analytic Applications

Business Intelligence
Sponsored by MicroStrategy

Data Integration

Database

Data Warehousing

E-Business

Enterprise Development

Scalability

Supply Chain

Archives

Database Programming

CIO Insight
January 30, 2001 Volume 4 Number 2

Email article



Join, Defray

Database performance problems can grow much faster than databases; choice of join technique can help

by Richard Winter

Many people have difficulty appreciating the magnitude of the scalability issues that arise as databases grow large. This is partly because data volumes are a bit like distances in outer space: The numbers are so large that we cannot understand what they mean. How many people seriously have any sense of the meaning of 10^{15}, the number of bytes in a petabyte? The very term "petabyte" seemed bizarre and unfamiliar only a few years ago.

Yet, at each of the six conferences I attended in the last two months, at least one keynote speaker uttered the word. All of a sudden, people are talking about petabytes seemingly with no fear -- even the fear they ought to have with regard to the scalability of their data management infrastructure.

Quick, tell me: How many megabytes in a petabyte? Answer: a billion. So 1.44PB is a billion diskettes. If you stack them eight to the inch, that's about 2,000 miles of diskettes.

That analogy doesn't answer any major questions about infrastructure, but it's a little like, "Toto, I don't think we're in Kansas anymore." A tornado may as well have picked all of us up from a world in which a high-density floppy disk stored a fair amount of data, dropping us moments later into a world where some companies have more data than you can store on all the floppy disks you can stack between Denver and my office in Waltham.

Simple Join Example

So if petabytes are more or less beyond comprehension, what do we do? I find myself thinking back to fundamentals. What I thought might be helpful to us all is an excursion into joins and scalability. It's nothing that involves really hard work for the reader -- we all have too much of that, right? -- just the highlights of what happens with joins as data volumes get large. I believe this will be helpful in grasping today's data volumes and getting a sense of what they mean for query processing in the data warehouse.

Joins are fundamental to relational database management. A simple example is the request: "List the names of customers who spent more than $1,000 on a single purchase in 1999." Assume that the database contains a customer table and a transaction table, where each transaction contains a customer ID. The simplest algorithm for performing this query is shown in Listing 1. This approach is

customer ID. The simplest algorithm for performing this query is shown in Listing 1. This approach is correct and works fine if the database is small.

Moderately Large Application

But let's work out an example where the database is the size of a modern, large-scale data warehouse -- not petabyte scale (I'll come back to that), just a pretty big one.

In a large business providing retail products or services, it would not be unusual to have data on 10 million customers and a billion transactions in the data warehouse.

Suppose that 20 percent of the transactions are from 1999 and that two percent of the transactions are for $1,000 or more. Suppose further that customer rows are 2,500 bytes long and transaction rows are 100 bytes long. Also, the database is built with a block size of 8,000 bytes (8KB), an increasingly common default block size today in data warehousing.

The customer table contains 25GB of data and the transaction table contains 100GB of data. Because these are 8KB blocks, you have about three million customer data blocks and about 12 million transaction data blocks. Typically, the customer data rows are not stored in any order.

If you take the previously described approach, you will read all the blocks in the transaction table once. You will find that 200 million of the transactions are from 1999. Of these, assuming no skew, you will find that two percent, or four million, are also purchases of more than $1,000. For each of these four million purchases fitting the criteria, you will, per the algorithm, "Get the matching row from the customer table." Whatever it takes to find such a matching row, it better not take a lot of work, because it needs to be done four million times!

If there are no indexes, you can scan the customer table each time until you find the matching customer ID. Assuming customer ID is the primary key, you can quit when you find the match. On the average, you would read half the blocks in the customer table before finding this match. That is, you will read 1.5 million disk blocks in each of the four million searches.

So that innocent sounding "Get the matching row from the customer table" involves running six trillion disk block reads by the time you have completed the join! Compared to this disk read work, you can ignore the effort of reading the transaction table and round off the cost of the operation to six trillion disk reads. These are sequential reads, so the disk drives will run at their maximum I/O speed, but it's still an awful lot of work. For example, if you charitably assume the disk drives will actually deliver 50 reads per second, you estimate 120 billion seconds of disk reading. It turns out that a billion seconds is about 32 years, so this task would take about 3,800 years of disk processing if done serially. During that time, disks would get a lot faster, but I think you can agree the join method described is not the approach you want to take in this case.

Small Application

Now, you may think this is a horrible join technique I have described. After all, if it would take 3,800 years to answer a simple query under plausible assumptions concerning a moderately large data warehouse, it must be a bad technique, right? Wrong. This is in fact a classic join technique: the nested loop join.

Many view it as the fundamental join technique in database management. And it works fine in this case if you have 1,000 customers and 100,000 transactions. Keep all the other assumptions the same and you will find that the customer table occupies 2.5MB and ought to be able to stay in the buffer pool even on a notebook PC. You would have 400 transactions of more than $1,000 in 1999, and would therefore do 400 main memory scans of the 1,000 customer records. The only disk I/O involved would be, in the worst case, to read the 10MB of transaction data and the 2.5MB of customer data, which requires about 30 seconds of disk I/O, using the same assumptions outlined earlier.

There are better join techniques even for this small database, but the point is that a standard technique, applied on a moderately large scale, doesn't just get a little bit worse. It explodes by requiring 3,800 years of disk processing instead of 30 seconds to handle this simple query. Again, this dramatic example just involves merely a moderately large database.

Worse Than Linear

Notice that this increase of query time is not proportionate (linear) with increase in database size. In the

first example, the sum of the size of the two tables is 125GB; in the second example, it is 12.5MB. The difference is a scale factor of 10,000. This is a formidable difference, larger than the difference between the floor space in a residential home and the floor space in one of the world's largest office towers. But multiply 30 seconds by the scale factor and you get 300,000 seconds -- approximately four days. Now, nobody wants to wait four days for the answer to a simple query, but this is not the biggest part of the scalability problem. The disk I/O time expanded not by a factor of 10,000 but by a factor of just under four billion!

So how can you do better? Well, you could reverse the process. For each customer, you could find the matching transaction. Then you would run the matching operation 10 million times. And each time, you would read an average of six million data blocks in the transaction table. That would take 60 trillion disk reads -- 10 times worse than the first approach!

Primary Index on Customer

You could try going back to the first approach and make the lookup process more efficient. For example, suppose there is a unique B-tree index on the customer table, on the customer ID column. Suppose a leaf in the B-tree index is 20 bytes (10 for customer ID, eight for row ID, and two bytes of overhead). Then the leaf level of the customer index occupies 200MB, and the entire customer ID index can be held in memory for the duration of the query in a large-scale data warehouse server.

After this index is in memory, the matching customer ID can be retrieved with a single disk read from the customer table. Now you have to run 12 million sequential disk block reads to scan the transaction table. And you have to run four million random reads from the customer table to find the matching customer rows. You have therefore reduced the total disk I/O requirement from six trillion to 16 million. You are doing 375,000 times less disk I/O. This is good. At 50 reads per second, you are now spending 240,000 seconds on disk I/O -- a mere three days, if done serially. It's still not what most users want for response time, but is a big improvement on the 3,800 you started with.

You got this improvement by reducing the cost of looking up a customer in the customer table. The good news is the big improvement it generated. The bad news is that this achievement is the best this method can deliver. As long as you keep looking up customers one at a time, you are going to need a random disk read for each one.

The Sort-Merge Join

In fact, the traditional approach to this join is to get rid of the row-by-row lookups. How do you do that? You sort both tables and then merge them, writing out from the merge only the data you need to complete the query. And, of course, you send into the sort only the rows that satisfy any predicates on the tables.

So you read the transaction table, which takes 12 million reads. But you need to sort only the transactions from 1999 greater than $1,000 (four million transactions). And all you need from them is the customer ID. So if the customer ID is 10 bytes, you need only 40MB of memory to hold the transaction data that goes into the sort. Then you read the customer table. That takes three million reads. But all you need from the customer row is the customer ID and the name -- say 50 bytes per row. So with 10 million rows, you end up with 500MB of customer data to sort. Sorting and then merging a total of 540MB of data is going to be small potatoes compared to the two huge table scans done in the earlier example to set up the join operation.

So the two table scans are all that really matter. The two table scans combined total 15 million disk reads -- a bit better than the last try, particularly because now all the I/O is sequential. In this case, the traditional sort-merge method results in performance that is more or less linear with the size of the tables to be joined. But the tables are big enough to make this improvement rather uninteresting.

A Clue

Still, we can gain insight of sorts from this classic join technique: The two tables contain 125GB of data, but only 540MB of that data -- less than one percent -- actually participates in the join operation. How do we know that? That is the amount of data that gets fed into the sort-merge operation. So to get scalability here, you need two things: (a) a way to get at that 540MB of data without doing 15 million disk reads and (b) an efficient enough way of operating on that 540MB so that it does not become a major factor in performance.

The more scalable join techniques used in today's relational database engines help accomplish these

two objectives, and in future columns I will discuss how. Still, some joins remain very expensive in large-scale operation, even with the best techniques. In upcoming issues, I will also explain how you can deal with the resulting database design issues for the large-scale data warehouse.

Richard Winter *(Richard.Winter@wintercorp.com), is a specialist in large database technology and implementation, and president of Waltham, Mass.-based Winter Corp. (http://www.wintercorp.com/).*

CSS IntelliVIEW

Copyright © 2002 CMP Media LLC,
a United Business Media Company
ALL RIGHTS RESERVED
No reproduction without permission
Intelligent Enterprise editors are not responsible for content appearing in Intelligent Enterprise Asia, a publication produced independently for the Asian market.